

ClientOpsCRM 2.0.0 Verification Report

Verification philosophy

A feature is considered implemented only when the shipped compiled path performs it against real persistence. The principal CRM test creates a new SQLite database and invokes the compiled executable as a subprocess for each operation. It then opens the same database directly for adversarial invariant tests. There is no mocked repository, fake CRM service, fake database or expected-output-only test.

Build gates

Three binaries are compiled from `crm/src/clientops_crm.cpp`:

- release: C++17, optimization, `-Wall -Wextra -Wpedantic -Werror`, stack protector, fortified libc use where supported;
- debug: C++17, debug symbols, same warning-as-error policy;
- sanitizer: C++17 with AddressSanitizer + UndefinedBehaviorSanitizer, frame pointers and warnings as errors.

The final verification evidence records SHA-256 hashes of compiled release artifacts and the toolchain versions used.

Behavioral and adversarial suite

The finalized suite contains 27 named checks. It is run independently against GCC release, GCC debug, GCC ASan/UBSan, and a separately compiled Clang 17 release binary:

1. version command after global options, without opening/creating the database
2. init and schema doctor
3. account normalization/uniqueness
4. contact email normalization/uniqueness
5. lead qualification
6. transactional lead conversion
7. failed conversion leaves no side effects
8. deal stage transition
9. deal win
10. closed deal cannot move
11. lost reason required
12. deal lost
13. task completion
14. double-close task rejected
15. note and custom field
16. invalid custom field key
17. FTS search
18. duplicate signal
19. dashboard/forecast JSON contracts
20. timeline/audit provenance
21. CSV content plus Relay JSON metadata round trip
22. Relay import idempotency
23. exports parseable
24. database adversarial guards
25. schema downgrade refused
26. deterministic repeated reads
27. final doctor

All four executable variants must report status: PASS and checks_passed: 27. Clang static analysis is also run independently and must emit no diagnostics.

Direct database adversary

The suite deliberately bypasses the application command layer. It attempts to insert a deal whose stage/status disagree and expects SQLite to reject it. It then attempts UPDATE and DELETE against an existing audit row and expects both to fail. This proves the relevant rules are not merely CLI conventions.

Import/export verification

CSV input includes a comma in a name, comma in company name, an embedded newline, escaped double quote, and JSON metadata. The imported record is read back and its JSON parsed for exact equality. The same CSV is replayed with --skip-existing; expected result is imported=0/skipped=1 and exactly one row for that Relay ticket.

JSON export is parsed and checked for the clientops-crm-export-v1 format marker. Contact CSV export is parsed with Python's CSV implementation and checked for records.

Determinism

After state stabilizes, the test issues the same deal list command twice and requires exact parsed JSON equality. The forecast uses stored integer values and probabilities; no clock/rand/remote model participates in the arithmetic.

Combined-system verification

The original Relay 1.2.1 C policy engine can be compiled/tested independently of the CRM. The combined launcher is shell-syntax checked; crm version, auto-initialization, crm doctor, JSON dashboard, and crm-build are exercised; and the Relay import bridge is shell-syntax and relational-contract checked against the shipped Relay schema.

The live Docker/PostgreSQL bridge requires a Docker host with the Relay stack running. Packaging evidence distinguishes executable CRM gates from any environment-dependent live-stack gate instead of converting an unavailable external runtime into a false PASS.

Release package audit

The packager excludes build scratch, runtime databases, caches, local secrets/environment files and dependency directories. Every delivered regular file must be non-empty. Inner ZIPs and the master ZIP are tested with the ZIP integrity checker. A SHA-256 manifest and file inventory are generated from the actual delivered bytes.